MAPLESURE INSURANCE · APPLICATION BRIEF

EnrolDirect

The online enrolment channel, the access question nobody had counted, and an analysis surface that refuses to overstate its own evidence.

Third repository in `repos/` · Python / FastAPI · S3 target for CR-2026-045 · Verified against a live run on 3 August 2026 · 24 regression tests passing

> THE ONE-LINE ANSWER
>
> *"EnrolDirect is the self-serve enrolment channel. It gates access on preferences a sponsor agreed at contract inception — and it carries a second surface that answers the question those preferences never anticipated: what to do with the people on the roster who hold no coverage yet. It computes that answer from configuration rather than asserting it, and it states plainly what the computation does not cover."*

Where it came from

The client shared an analysis spike from their own estate: two online-enrolment access controls, a third population that fitted neither, and a list of questions to answer before anyone wrote code. The spike's deliverable was a document.

EnrolDirect is that scenario rebuilt as a **running application**, fully de-identified — fictional sponsors, invented applicants, MapleSure system names throughout. Nothing from the client's systems, people or issue keys appears in the code, the data, or this brief. The interesting move is that the analysis is not a document here: it is three endpoints that recompute themselves from whatever the configuration currently says.

CR-2026-045 was rewritten on 3 August 2026 to follow that same source material — business objective, target member type, and acceptance criteria in given-when-then — so the change request reads as the client's own story rather than ours. Its technical acceptance criteria were not touched, which is why the committed recording still replays against it.

What the application does

Access to the self-serve channel is gated by two preferences the plan sponsor agrees at contract inception. They live on the contract record in **PolicyCore**; EnrolDirect enforces them and does not own them. A change to what a preference *means* is therefore a change to a contract between two systems, not a local edit.

ACCESS PREFERENCE	WRITTEN FOR
`Online Enrolment – Member`	People already holding an active benefit under the contract, changing their own coverage.
`Online Enrolment – Guest`	People with no active benefit who still have reason to enrol — retiree continuations, spousal transfers, sponsor-agreed exceptions.

The gate runs three checks, in this order

1. **The contract is active.** A lapsed contract keeps whatever preferences it was configured with, so this runs first — reading preferences off a lapsed contract would let stale configuration grant access.
2. **The applicant's category resolves to a preference.**
3. **The sponsor enabled that preference.**

Decisions carry their reasons, not just a boolean. A denial that cannot say which gate closed becomes a support ticket, and avoiding a wave of those is most of why the classification question was worth answering deliberately.

> **ORDERING IS THE FRAGILE PART**
>
> Gates 1 and 3 are reversible without breaking a single category-level test, and reversing them reintroduces the stale-configuration hole silently. That is why the regression suite asserts the ordering directly rather than only asserting outcomes.

Three populations, two preferences

Two categories map cleanly onto a preference because the preferences were designed around them. The third does not — and it is not a halfway state on the way to becoming a member. It is a population the sponsor has already accepted onto the roster who has not taken up coverage.

CATEGORY	ON ROSTER	ACTIVE BENEFIT	PREFERENCE	SEEDED
MEMBER	yes	yes	Member	4
GUEST	no	no	Guest	2
PROSPECT	**yes**	**no**	NONE	6

Treating a prospect as a guest — a stranger to the contract — is uncomfortable. Treating them as a member — someone with coverage to change — is inaccurate. As checked in, **the application picks neither**: a prospect resolves to no preference and is refused at the second gate. The analysis surface evaluates both options as parameters of its own computation, without the runtime gate resolving anyone through either of them.

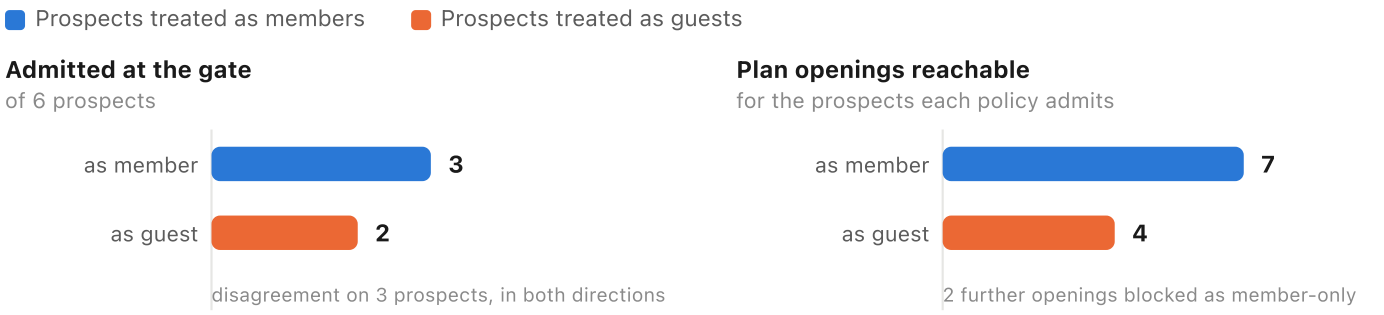
CR-2026-045 settles it, and settles it as a **single module-level value** in `applicants.py` rather than a per-call parameter. That is deliberate: the gate enforces one policy for the whole book, and a caller that could pass its own would be choosing its own access rules.

The classification bites twice

The obvious effect is at the gate: how many prospects get in. The second effect is past it, in the catalogue — some plans attach to existing coverage and are open to members only, so a prospect admitted *as a guest* also reaches a smaller catalogue. The two effects run in opposite directions, and reporting only the first would understate the gap between the policies.

What each policy changes, measured against the seeded book

Left: prospects admitted at the gate, of 6. Right: plan openings those admitted prospects can reach.



Reach is counted only for prospects the gate admits — a plan you cannot reach because you were refused at the door is already a denial.

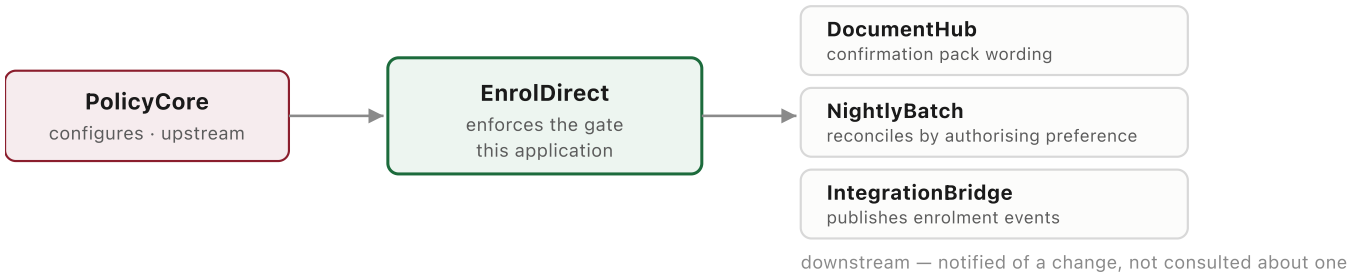
Both figures use the same colour for the same policy. Colour follows the entity, never its rank within a chart, and every bar carries its value directly so identity never depends on colour alone.

The analysis surface

Three endpoints, all pure functions of the seeded directory and the real eligibility rules. **No model call anywhere in them** — so there is no cache key, nothing to warm, and nothing that can be confidently wrong the way generated prose can. Same reasoning that keeps `diagram.py` and `acceptance.py` deterministic on the S3 side.

ENDPOINT	ANSWERS
`/api/analysis/consumers`	Which systems read or write these preferences, and in which direction — upstream, enforcing, downstream.
`/api/analysis/preference-usage`	Which contracts enable which preference; headcount per applicant category.
`/api/analysis/prospect-impact`	Both policies' grant counts, catalogue reach, every applicant they disagree about, and a recommendation.

The dependency map



Declared, not discovered. EnrolDirect cannot see the other systems' code, so claiming to have derived this by inspection would be a lie. It is the analysis's stated input, and it is wrong the moment a consumer appears without the table being updated — which the risk list and the UI both say out loud.

The recommendation, and what it refuses to claim

The application recommends **Guest**: it is the narrower grant, a prospect holds no coverage for the member screens to show, and widening access later is easier to justify than withdrawing it. The

recommendation explains the configured default rather than deriving it from the counts — a recommendation that flipped with the seed data would be curve-fitting, not policy. What the counts do is *size* it.

> **IT REPORTS ITS OWN COST**
>
> Two seeded prospects are denied under the recommended policy that the alternative would admit — both on contracts whose sponsor enabled member access only. Those sponsors would have to enable guest access for their prospects to self-serve. That number is on the same screen as the recommendation, not in a footnote.

Not evidenced by this analysis

Four consequences are real and not computable from configuration data. They are named in the payload, rendered in the UI, and asserted by a test — because an analysis that lists only the numbers supporting its recommendation is advocacy, and this one feeds a decision owner.

- **Regulatory position.** Whether a prospect admitted under member access has been represented as a plan member is a compliance question. No count answers it.
- **Support load.** Denied prospects call the service desk; admitted prospects reaching member screens with no coverage to display also call. The seed cannot say which volume is larger.
- **Downstream subscriber behaviour.** The consumer list is declared. A subscriber filtering on the authorising preference that nobody recorded will change behaviour unannounced.
- **Prospect expectation.** What the sponsor told these people they would be able to do is not in the contract record.

The decision owner is named explicitly as plan administration jointly with compliance. The application's stated position is that it sizes the options and does not have standing to choose between them.

Running it

```
apps/run-enroldirect.sh          # http://localhost:8083/
```

Standalone. It seeds contracts, applicants and plans in-process and calls no other service, so it needs nothing running beside it and nothing beyond the venv — FastAPI and uvicorn were already pinned. Port comes from `ENROLDIRECT_PORT` in `.env`, defaulting to 8083 (8081 and 8082 belong to ClaimsPortal, 8501 to PolicyCore). It survives a port to a locked-down environment: no cloud services, no Docker, no OS-specific paths.

SCREEN	WHAT IT IS FOR
Overview	Estate counts, both comparison figures, and the recommendation with its cost.
Enrol	The real journey: identify, see your catalogue, pick a tier, submit. Unavailable plans stay listed with their reason.
Access check	Run the gate for any applicant under either policy and read the reasons back.
Contracts & plans	Every contract, its enabled preferences, and the plans behind them.

SCREEN	WHAT IT IS FOR
Preference analysis	Impact, catalogue reach, preference usage, consumers.
Audit log	Every attempt, successful or refused, with the preference that authorised it.

Modules

FILE	OWNS
`preferences.py`	The two access preferences and the PolicyCore integration contract.
`applicants.py`	The three categories, the prospect policy, and the validation that a MEMBER without coverage is a data fault.
`directory.py`	Seeded contracts and applicants, shaped so the two policies genuinely disagree.
`benefits.py`	Plans, coverage tiers, and `memberOnly` — the second bite.
`eligibility.py`	The three-gate access check and the effective-category resolution.
`enrolments.py`	Submission, the four refusal codes, and the audit store.
`impact.py`	The whole analysis surface. No model call.
`main.py`	FastAPI wiring and the payload shapes.

What the tests protect

`tests/test_regression_enroldirect.py` — 24 tests, checked in, human-authored, and named by no target's codegen or testgen allowlist. It lives in the top-level `tests/` rather than under `repos/enroldirect/` because anything ending `.py` under a repo root joins the codegen candidate pool, and a suite the pipeline can rewrite is not an independent check of anything. `tests/test_s3_testrun.py` asserts that separation rather than trusting it. Every one of these must pass **before and after** CR-2026-045 — they are invariants, not assertions about the change.

WHAT IS ASSERTED	WHY IT NEEDS A TEST
The contract gate runs before the preference gate	Reversing them lets a lapsed contract's stale preferences grant access, and every category-level assertion still passes.
The prospect policy never moves a member's or guest's outcome	That is the promise the reclassification makes to every other consumer of these preferences.
The two policies genuinely disagree, in both directions	A comparison reporting two identical columns looks like agreement and is in fact a broken parameter.
Nothing the gate refuses can be enrolled by any route	The enrolment path reuses the gate rather than reimplementing it; this is the observable consequence.
Catalogue reach counts only prospects the gate admits	Otherwise a prospect refused at the door is counted twice and the gap is inflated.
Unavailable plans are marked, not hidden	A plan that silently vanishes generates the support call an explained one does not.

WHAT IS ASSERTED	WHY IT NEEDS A TEST
The analysis states what it does not establish	The honesty clause is load-bearing, so it is enforced rather than trusted.

Now an S3 target — and the one that is shaped differently

EnrolDirect used to be described here as deliberately *not* a target: the code was on disk, no target was registered, and the console correctly reported `automation_available=False`. **That is no longer true.** It is registered as `enroldirect-prospect-access`, running CR-2026-045, with its own committed replay recording. Routing and automation are both live for it.

> **ITS BASELINE IS A REMOVAL, AND THAT IS THE WHOLE POINT**
>
> *"The other change requests add something — a field, a tier, a deductible. This one settles a question. What is checked in is the state after the impact analysis and before anyone acted on it: a prospect resolves to no preference and is turned away. The 'before' is not a missing feature, it is a refusal nobody decided on."*

The read-only core file

`impact.py` is in the target's **core files** — relevance always recalls it into the prompt — and deliberately **not** in its codegen allowlist. Both halves are needed. The model has to read the analysis to understand what the CR means; it must not edit it, because the analysis has to keep sizing *both* options after one is adopted. A comparison reporting only the option in force cannot answer "what is this costing us", which is the question that reopens the decision. Core recall gets it in; the allowlist keeps it out of the diff, and this target carries its own file-set validator that fails loudly if a read-only file comes back modified.

TARGET PROPERTY	VALUE
Target id	`enroldirect-prospect-access`
Cache namespace	`enroldirect_prospect_access`
Change request	`crs/CR-2026-045.md`
Board ticket	`AMS-1045`
Editable core files	`applicants.py · eligibility.py · enrolments.py · main.py`
Read-only core file	`impact.py`
Generated suite	`tests/test_s3_prospect_access.py`
Regression suite	`tests/test_regression_enroldirect.py`
Reset	`demo/reset_s3_enroldirect.sh`

AMS-1045 was never hand-seeded. `crs/CR-2026-045.md` exists, so the board opens a ticket for it automatically, keyed off the CR identifier and landing unassigned — which is what puts it in front of the manager. The rest of the onboarding contract, including the `.s3targets.json` manifest a dropped-in repository ships, is in `repos/README.md`.

> **VERIFY THE "BEFORE" BEFORE YOU PRESENT IT**
>
> `POST /api/eligibility/check` for a prospect (AP-4003 or AP-4008) must come back `"granted": false` with `"requiredPreference": null` and the reason *"category PROSPECT has no online enrolment preference"*. A prospect refused for any *other* reason means the reset did not take, and the beat's opening claim is then wrong on screen.

What it deliberately is not

Not the same thing as PolicyCore's enrolment subsystem

MODULE	QUESTION IT ANSWERS
`repos/policycore/enrolment/`	*When* may this person join? Waiting periods, eligibility dates, life events, dependant age-out.
`repos/enroldirect/`	May this person use the *online channel* at all? Access preferences, categories, the gate.

They compose and neither imports the other. Someone can be eligible to join and have no online access, or the reverse. Merging them produces one function answering two questions that can only report one reason.

Not persistent

The enrolment log is in-process and resettable from the UI. Nothing survives a restart, and nothing is supposed to — the application has to run in a locked-down sandbox, so a datastore is a dependency it cannot take.

> **DATA**
>
> All synthetic. Plan sponsors are fictional employers, every applicant name is invented, and nothing derives from a real roster. The source material that prompted this build is excluded from version control.

MapleSure Insurance · `repos/enroldirect/` · Companion to `repos/enroldirect/README.md` and `repos/README.md`. Every figure on this page was re-measured against a live run on 3 August 2026 — `/api/analysis/prospect-impact`, `/api/contracts` and `/api/eligibility/check` on :8083 — not carried forward from an earlier draft; re-measure again before quoting them after a seed change.